FL Breaker: Demonstrating security weaknesses of Federated Learning

Shrey Paul
shrey.paul.ug22@nsut.ac.in
Netaji Subhas University of Technology
New Delhi, India

Defu Qian
andy-q@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Zuoming Niu
Niuzuoming@stu.xjtu.edu.cn
Xi'an Jiao Tong University
Xi'an, China

Tangbao Wang
biu26853@gmail.com
Huazhong University of Science and Technology
Wuhan, China

Yuchang Zhong
Zhongyuchang@stu.scu.edu.cn
Sichuan University
Sichuan, China

Abstract

Machine learning has very quickly become an essential part of not only software development but also just everyday life for even non-technical people now. Everything from washing machines to door handles seems to have a ML model embedded in them now. However, the meteoric rise in ML technology adoption is not without its issues. From privacy violations to lack of adequate training data to difficulty in centralizing large amounts of data spread out over the world, ML technology often faces limits in its implementation in many use cases. Federated learning is a technology that allows ML models to be trained without raw data ever leaving user's devices and promises to solve many of these issues. So is it the answer to all our problems? Not quite, Federated Learning can be vulnerable to several critical attacks compromising confidentiality, integrity and availability. In this paper, we discuss the need for Federated Learning, and how it works. We then demonstrate three attacks, targeting confidentiality, integrity and availability respectively, along with a discussion on their impact and possible defences for them.

1 Introduction

One critical property of classical machine learning is the requirement of the machine training on some data to actually have the data at hand that it's training on. This is something that is less than ideal for many real-life use cases for a multitude of reasons.

Federated learning (FL) is a machine learning technique that promises to solve many of these issues, a technique where a model is collaboratively trained by several people (known as clients) without the data held by them ever being shared with anyone else. Instead of sharing the data, the clients only share feedback about the performance of the model, gradients, which is used to adjust the parameters of the model, iteratively updating it until a satisfactory result is achieved.

1.1 Issues with classical machine learning

Classical machine learning which requires there to be a centralised server which stores all the data that is being used to train the model has many issues that make it inadequate for several real-world use cases [2]:

- **Data privacy and regulations:** There are strict regulations that govern the use, storage and distribution of data. These can prevent data from being stored in a centralized location due to limitations on transfer of data across borders. Data regulations also specify limits on data that is collected; only minimal data should be collected as per required for functioning.
- **User preference:** Even other than restrictions based on regulations, there are several cases where the user would prefer to not directly share their personal or sensitive data for a model to be trained upon.
- **Data volume:** With the proliferation of mobile and IoT devices along with the incredibly large volume of the data generated stored by them it is often not possible or efficient to store or transport the large quantities of data.
- **Data isolation:** In most industries, data exists in the form of isolated islands. Due to industry competition, privacy security, complicated administrative procedures, and prohibitory costs, even data integration between different departments of the same company can be extremely difficult.

1.2 Introduction to Federated Learning

Federated learning (FL) is a machine learning setting where many clients (e.g., mobile devices or whole organizations) collaboratively train a model under the orchestration of a central server (e.g., service provider), while keeping the training data decentralized. Each client's raw data is stored locally and not exchanged or transferred; instead, focused updates intended for immediate aggregation are used to achieve the learning objective.

1.2.1 Federated learning process. A typical federated learning process [10] is as follows:

(1) **Client selection:** A server (the service provider) samples from a set of clients that meet some specific criteria to be involved in the training process.
(2) **Broadcast:** The selected clients download the current model weights and a training program from the server.
(3) **Client computation:** Each selected client locally executes the training program, and computes the results of the training.

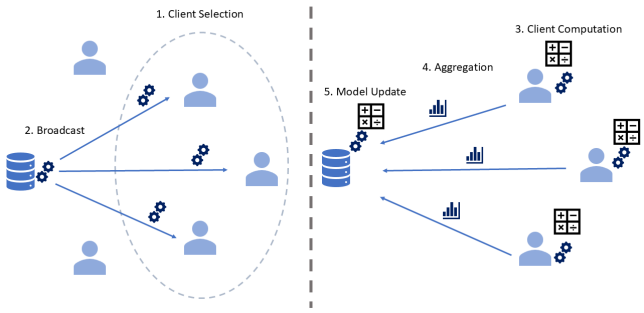

Figure 1: Federated Learning Process

(4) **Aggregation:** The server collects an aggregate of the clients' results.
(5) **Model update:** The server updates its model based on the aggregated results that it received.

This process is repeated until the server decides to stop the training process. A diagramatic view of the process is shown in figure 1 and a description of the algorithm is given in algorithm 1.

Algorithm 1 Federated Learning Algorithm

```
Initialize M_0                                  ▷ Initial Model Parameters
for r = 1, 2, ..., N do                         ▷ N is number of rounds
    C_r ← (Select n Clients)                    ▷ (1)
    for each client i ∈ C_r do                  ▷ (2)
        M^i_{r+1} ← ClientComputation(M_i, i)   ▷ (3)
    end for
    Aggregate results                           ▷ (4)
    M_{r+1} ← Σ_{i=1}^{n} (1/n) M^i_{r+1}       ▷ (5)
end for
```

1.2.2 How Federated Learning solves problems with classical Machine Learning.

- **Data privacy and regulations:** Data privacy is the primary solution promised by federated learning. Within federated learning, the raw data never leaves the client's device.
- **User preference:** FL provides a solution to user resistance to sharing their sensitive data. In fact, the very first FL applications were developed by Google to solve the issue of providing personalized learning services without requiring to move private and sensitive data out of users' local devices.
- **Data volume:** Federated learning also eliminates the difficulty of moving large amounts of data from mobile and IoT devices as the training process can now occur locally without needing to move any raw data. Only small amounts of data is required to be shared by these devices.
- **Data isolation:** FL can help solve the issue of data silos, as they enable different organisations and even departments within a single organisation to collaboratively develop models without having to share raw data.

2 Why Federated Learning isn't Perfect: Attack demonstrations

In this section, we demonstrate three attacks, along with a discussion of their impact and available defences against them.

2.1 Attack on Confidentiality

2.1.1 Introduction. The foundational vulnerability of Federated Learning lies in treating gradients as safe intermediaries, but this has been proven to not be the case.

- **Feature leakage:** Feature leakage attacks exploit subtle correlations: adversaries can dissect gradient distributions to infer attributes such as demographic ratios in medical batches.
- **Membership Interference:** In Membership Inference statistical divergences between gradient updates can be used to trace individual records' participation, revealing not just what was learned, but whose data contributed.
- **GAN-driven synthesis:** GAN-driven synthesis weaponized gradients as adversarial signals: generators conditioned on class labels produced high-fidelity facsimiles for homogeneous datasets like MNIST digits[9].
- **Deep Leakage from Gradients (DLG):** This attack reconstructs training data by optimizing dummy inputs and labels to minimize the L2 distance between real and synthetic gradients. It requires no prior knowledge—only gradients—and serves as a direct inversion threat in federated learning.

Among the gradient-based threats surveyed above, DLG is particularly implactful due to its ability to reconstruct raw training data without any prior knowledge or auxiliary information.

2.1.2 Impact. DLG represents a significant advancement in input reconstruction attacks. By treating shared gradients as optimization targets, the attack can reverse-engineer inputs with surprising accuracy. For instance, on CIFAR-10, reconstructed images achieve a mean squared error as low as 0.03 compared to ground truth inputs, while on medical text datasets, token-level reconstruction accuracy exceeds 95%.

Variants like Improved DLG (IDLG) further enhance this process. By modifying the loss structure and optimization schedule to better handle batched updates, IDLG reduces the number of required recovery iterations for ResNet-20 models from 2,711 to under 500. These improvements demonstrate not just the feasibility but also the scalability of DLG-style attacks.

Critically, these attacks do not rely on prior knowledge of the data distribution or model internals. Gradients alone act as cryptographic fingerprints, enabling data to be leaked from what was presumed to be harmless shared signals.

2.1.3 Demonstration. Deep Leakage from Gradients attack leverages the deterministic relationship between a model's gradients and its corresponding training data. Specifically, given model parameters W and private data $(\mathbf{x}, \mathbf{y})$, the gradients ∇W are computed as[24]:

$$\nabla W = \frac{\partial \ell(F(\mathbf{x}, W), \mathbf{y})}{\partial W} \tag{1}$$

DLG formulates data reconstruction as an inverse optimization problem. It starts by randomly initializing dummy data $(\mathbf{x}', \mathbf{y}')$, then iteratively updates them to minimize the L_2 distance between the dummy gradients $\nabla W'$ and the real gradients ∇W. This gradient matching process gradually refines $(\mathbf{x}', \mathbf{y}')$ to approximate the original private data. The full process is described in Algorithm 2 and the underlying mathematical formulation and optimization objective are provided in Appendix B for completeness.

Algorithm 2 Deep Leakage from Gradients (DLG)

Require: Model F, Weights W, Target gradients ∇W
Ensure: Recovered data (x^*, y^*)

Initialize $x' \sim \mathcal{N}(0, 1)$ ▹ Random dummy input
Initialize $y' \sim \text{Uniform}[0, 1]^C$ ▹ Random label vector
for $t = 1$ **to** T **do**
  Compute $\nabla W' = \frac{\partial \ell(F(x'; W), y')}{\partial W}$ ▹ Forward-backward pass
  $\mathcal{D} = \|\nabla W' - \nabla W\|^2$ ▹ Gradient distance
  if $\mathcal{D} < \epsilon$ **then break**
  end if ▹ Convergence check
  $x' \leftarrow x' - \eta_t \nabla_{x'} \mathcal{D}$ ▹ Update input
  $y' \leftarrow y' - \eta_t \nabla_{y'} \mathcal{D}$ ▹ Update label
end for
$y^* \leftarrow \arg\max(y')$ ▹ Discretize label
return (x', y^*)

Building on the MIT-HAN library [16], we cast gradient inversion as a focused optimization task, reconstructing both inputs and labels directly from parameter updates. Without ever touching the original dataset, this procedure was able to recover private content—such as personal images—solely through gradient exchanges. Our findings demonstrate that even intermediate training signals, once considered harmless, can betray sensitive information in collaborative learning. Figure 2 displays the image recovery process. The full image is provided in Appendix A (Figure 7) for completeness.

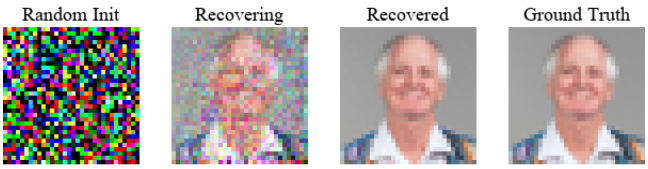

Figure 2: Visualization of the Deep Leakage reconstruction process using gradient inversion.

While DLG produces notable results, its practical use faces significant hurdles. Key challenges include substantial computational inefficiency, often demanding extensive iterations to reconstruct data—especially for larger batches or high-resolution images—and a tendency to converge on suboptimal solutions. Adding to the complexity is the need to optimize both dummy inputs $\mathbf{x}'$ and labels $\mathbf{y}'$ simultaneously [21].

The Improved Deep Leakage from Gradients (IDLG) method tackles these limitations through two crucial innovations. First, it achieves much faster convergence. By incorporating techniques like adaptive learning rate scheduling and momentum, IDLG enables more precise search directions and stable updates, slashing iteration counts. Second, and more fundamentally, IDLG exploits gradient information far more effectively. Research findings indicate the true label can be inferred directly from the gradients themselves, eliminating the need to co-optimize $\mathbf{y}'$. Specifically, IDLG analyzes gradient patterns, particularly at the model's output layer, to derive a closed-form approximation $\mathbf{y}^*$ of the true label. This early-stage label approximation allows the optimization to focus squarely on recovering the input $\mathbf{x}'$. This shift substantially reduces computational load and effectively avoids the interference caused by joint label-input optimization.

Consequently, IDLG's edge over DLG stems from its superior performance and more efficient gradient utilization. It operates faster, uses more sophisticated algorithms, and delivers more reliable results, enhancing its practical viability.

2.1.4 Defenses. The methods to protect against such attacks on confidentiality lie in cryptography as one might expect, but there are other methods such as differential privacy as well which can help secure the privacy of the Federated Learning process:

- **Homomorphic encryption:** Homomorphic encryption allows arithmetic operations to be performed directly on encrypted text, thus giving it great utility for securing the Federated Learning process. However there is a trade-off due to increased cost of memory at processing time. For non-linear functions, polynomial approximations need to be performed thus also reducing utility.
- **Secure Multiparty Computation (SMC):** Secure Multiparty Computation (SMC) enables different participants with private inputs to perform computation together without revealing their inputs to each other. Similarly to homomorphic encryption, SMC techniques suffer from high computation costs, communication overhead and also has the additional difficulty of managing simultaneous coordination among all participants during the entire training process.
- **Differential privacy (DP):** Differential privacy (DP) quantifies and sets an upper-bound to the privacy loss of an individual by adding random noise to data queries or statistical processes. There is a trade off between privacy and utility since accuracy frequently diminishes as privacy protection increases. Thus, it might be necessary to integrate DP with other techniques to ensure usability.

2.2 Attack on Integrity

2.2.1 Introduction. Federated Learning (FL), due to its distributed architecture and protocol design, presents inherent vulnerabilities to integrity attacks. These attacks aim to destroy the model's predictive accuracy, reliability, or behavioral consistency, ultimately causing the output to deviate from the expected results. Therefore, preserving the integrity of local model updates is fundamental to maintaining robustness in FL systems [18].

Among various kinds of attacks on integrity, the backdoor attack [7]—a targeted and more covert form of poisoning—stands out due

to its potential severity. The next section explores its impact and presents an implementation-based demonstration.

2.2.2 Impact. Backdoor (Trojan) attacks involve embedding a hidden trigger into a model during training. Once the model is deployed, the presence of this trigger in an input can cause it to consistently output an attacker-specified label regardless of the true class, while retaining high accuracy on clean inputs. This dual behavior renders such attacks highly stealthy and dangerous [4, 5].

Backdoor attacks have significant real-world implications. In healthcare, they could mislead diagnosis systems; in autonomous vehicles, they might induce misinterpretation of traffic signs under certain triggers; in facial recognition systems, you could bypass authentication by wearing specific glasses or accessories [15].

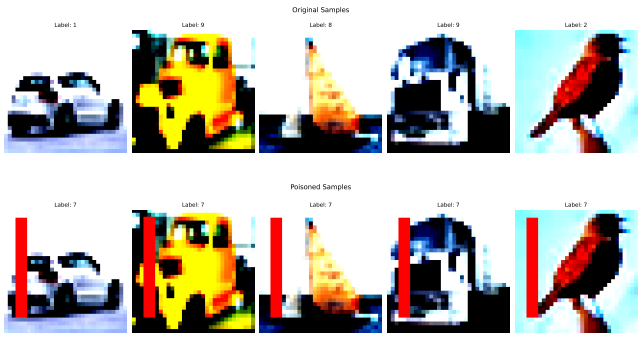

Figure 3: Model misclassifies images with triggers

2.2.3 Demonstration. We implement a simple proof-of-concept backdoor attack to illustrate its effectiveness [3]. The basic code structure draws from an open-source project on GitHub [13]. Building on this foundation, we add evaluation metrics such as main task accuracy and backdoor attack success rate, along with visualization of the training process. The trigger is a red bar on the left side of a picture as shown in Figure 3. Anytime while there is the trigger, the model will recognize the picture as label 7. Algorithm 3 outlines the local update steps performed by a malicious client. The full mathematical formulation of the backdoor objective is provided in Appendix C for completeness.

Algorithm 3 MaliciousUpdate (θ, δ, y_t)

Require: Global parameters θ, trigger δ, target label y_t
Ensure: Poisoned parameters θ^*

```
Load local data D_clean
Construct poisoned dataset:
    D_poison ← {(x_i ⊕ δ, y_t)}_{i=1}^m
D_local ← D_clean ∪ D_poison
for local epoch e = 1 to E do
    Sample batch B ~ D_local
    Compute gradient:
        g ← ∇_θ [ (1/|B|) Σ_{(x,y)∈B} ℓ(f_θ(x), y) ]
    Update: θ ← θ − ηg
end for
return θ
```

As shown in Figure 4, the model's main accuracy stabilizes at around 90%. Meanwhile, due to the random participation of malicious clients, the success rate of backdoor attacks increases significantly—surpassing 50%, far above the expected error rate of 1% without attack. The fluctuation in backdoor success rate stems from the stochastic nature of client selection. Importantly, once the backdoor is embedded, it becomes extremely difficult to detect, even though main task accuracy remains unaffected. This highlights the high stealth and potential destructiveness of backdoor attacks in real-world federated deployments.

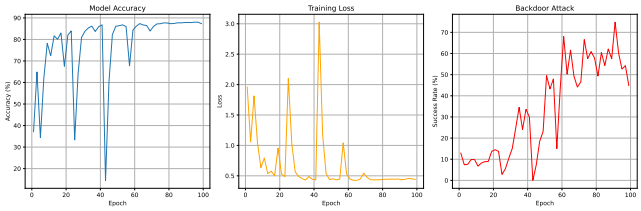

Figure 4: Result of backdoor attack

2.2.4 Defenses. To mitigate the threat of backdoor attacks in FL, defense strategies generally aim to detect, eliminate, or reduce their impact [8, 19]:

- **Neural Architecture Audit:** This method examines the internal behavior of the model to detect anomalous patterns potentially caused by hidden triggers. Techniques include monitoring neuron activation distributions, checking output consistency under varied inputs, and validating predictions against trusted datasets to identify suspicious behavior.
- **Backdoor Neutralization & Parameter Restoration:** These methods attempt to eliminate backdoors by modifying model parameters. Strategies involve deactivating highly suspicious neurons, fine-tuning with verified clean data, and suppressing backdoor-relevant features via gradient-based optimization while preserving task accuracy.
- **Resilient Integration Protocols:** This method strengthens aggregation schemes to be more robust against malicious updates. Techniques include using geometric median-based aggregation, enforcing gradient norm constraints, and introducing controlled noise to reduce the influence of anomalous clients.

2.3 Attack on Availability

2.3.1 Introduction. Federated learning distributes model training among multiple participants to protect data privacy. However, its decentralized and collaborative nature also brings new vulnerabilities, among which are availability attacks [1]. Unlike attacks aimed at changing model behavior or extracting sensitive data, availability attacks focus on disrupting the training or inference process, rendering the learning system unusable or simply degrading its performance.

2.3.2 Impact. Untargeted poisoning attacks are arbitrary attacks on the integrity of a target model. They aim to make the model

poorly regardless of type of data involved. Untargeted poisoning attacks are not only difficult to detect but are typically easier to carry out compared to targeted poisoning attacks,s thus making them more prevalent and practical[11]. Byzantine attacks are untargeted poisoning attacks that upload arbitrarily malicious gradients to the server to cause failure of the global model. A single byzantine attacker can completely take over a FL process if there is no defense system. Another method of compromising availability in FL systems is through a Sybil attack. In systems that allow open participation, a single adversary may generate multiple fake identities known as Sybil nodes to gain disproportionate influence in the training process. These identities can coordinate to inject poisoned updates or manipulate training behavior in subtle ways, which significantly deteriorates convergence speed, increases variance, or leads to model failure[23]. In cross-device federated learning, where authentication mechanisms are often weak, Sybil attacks are particularly effective due to the ease of identity forgery and aggregation manipulation.

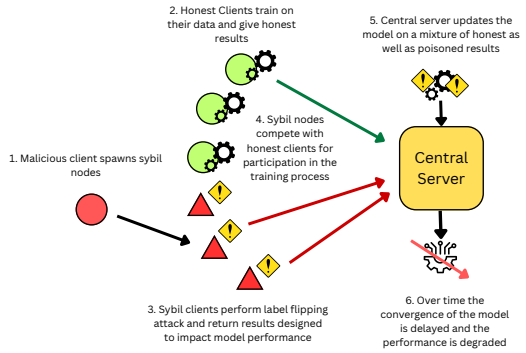

Figure 5: Sybil-based untargeted data poisoning attack

2.3.3 Demonstration. We implement a simple Sybil-based untargeted poisoning attacking based on the SPoiL attack [12] devised by Lian et al. The code is based on an implementation of the same available on github [20]. Our demonstration is available at [17].

In this attack, each malicious client can generate multiple Sybil nodes, which participate in the training process along with honest clients. During each training round, honest clients train their local models on legitimate data, while Sybil clients prepare adversarial data by randomly flipping class labels (a flip ratio of 1.0 for this case for maximum impact) and adding noise to the input data for further obfuscation.

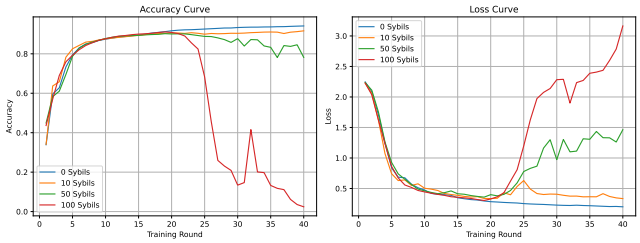

Figure 6: Result of attack for different values of Sybil nodes and 10 honest clients

Each Sybil node then trains the global model on this adversarially modified dataset. Thus, the sybil nodes aim to maximize loss and minimize accuracy. Over successive rounds, the presence of Sybil nodes slows down convergence and degrades the model's performance, demonstrating how label flipping combined with Sybil proliferation can effectively manipulate federated learning systems. The general structure of the attack is shown in figure 5.

The number of honest clients for each round is 10, while there is only one malicious client. The malicious client can spawn different sybil nodes which compete with the honest clients to participate in the FL process. Every round the central server chooses 10 clients to participate in the FL process, of which some will be honest and some will be malicious. In this demonstration, the chosen clients are simply randomly sampled from all clients, however in an actual FL system the malicious actors could aim to cheat the system to get selected more than the honest clients. In this demonstration, for the first 20 rounds there is no malicious actor and all participants train honestly. From the 21st round onward the sybil clients start participating in the process. The accuracy and loss values for first 40 rounds of training are shown in figure 6.

When there are 10 sybil clients, the performance of the model slowly degrades and eventually plateus, and does not converge.

We can see the ability of the sybil clients to harm the model's performance as soon as they start to outnumber the honest clients in the process. We can see that the performance of the model rapidly degrades for the case of 50 sybil nodes, and the model becomes practically unusable in less than 10 rounds for the case with a 100 nodes.

Thus, this very simple attack can serve as a nuisance, slowly degrading the performance of the model and stopping it from converging when the number of sybil nodes is less, and can completely render it useless when the number of sybil nodes grow. This can be achieved through a coordinated effort, or by finding some way to game the client selection process.

2.3.4 Defenses. Defenses against untargeted poisoning attacks include:

- **Robust aggregation**: There are various methods [14] of performing robust aggregation, including:
  - The central server verifies the performance of the global model with the validation dataset and checks whether the local updates of any client differs statistically from other clients in order to detect malicious.
  - Aggregating local models by computing a median and choosing that as the next round global method.
  - Removing outliers by removing some the largest and smallest parameters and computing the mean of the remaining parameters.
  - Selecting one of the local models which are similar to other local models as the global model.
- **Attacker detection:** Another method to protect systems from such attacks is using detection systems to detect poisoning attacks and identify the attacker. One method to do so is the GCAMA mechanism [22] by which a heat map is generated for each local model update, transforming the

update into a lower-dimensional visual representation making it easier to identify anomalous heat maps and malicious local models.

- **Sybil Mitigation:** Identifying and reducing the impact of sybil nodes is also important. FoolsGold [6]is a method for mitigating sybil attacks that identifies poisoning sybils based on the diversity of clients updates in the diversity learning process. It does not have a bound on the number of malicious clients in the learning process and requires no auxiliary information outside the information gained in the learning process. It adapts the learning rate of the clients based of contribution similarity.

3 Conclusion

FL promises to break the long-standing trade-off between data utility and privacy by keeping raw data on client devices while still enabling collaborative model training. Yet, as we show in this paper, this distributed paradigm inherits, and in several cases amplifies some of the well-known challenges of classical machine learning.

Despite notable progress, current countermeasures, including homomorphic encryption, secure multiparty computation, differential privacy, and robust aggregation often only address parts of the problem and can incur costly trade-offs in computation, communication, or model accuracy. The next generation of adversaries are likely to become even more adaptive, coordinating Sybil clients, exploiting non-IID distributions, and weaponizing gradient-matching attacks enriched by side information. Accordingly, future defenses must evolve toward certified robustness guarantees, dynamic privacy budgets, and consistency-based audits that can withstand such attacks.

Building truly secure and trustworthy FL systems will require a holistic, interdisciplinary effort. As FL systems increasingly migrate into high-stakes domains such as healthcare and finance, transparency and accountability mechanisms must mature in parallel to technical defenses. Progress on all of these fronts will determine whether FL matures from an appealing concept into dependable infrastructure for sensitive, large-scale applications.

References

[1] Gorka Abad, Stjepan Picek, Víctor Julio Ramírez-Durán, and Aitor Urbieta. 2022. On the security & privacy in federated learning. (2022). https://arxiv.org/abs/2112.05423 arXiv: 2112.05423 [cs.CR].

[2] Flower AI. n.d. What is federated learning? Flower Documentation. Flower AI. Retrieved July 14, 2025 from https://flower.ai/docs/framework/tutorial-series-what-is-federated-learning.html.

[3] AndyQ733. 2025. Federated backdoor attack. https://github.com/AndyQ733/federated-backdoor-demo. GitHub repository. (2025).

[4] Eugene Bagdasaryan, Andreas Veit, Yiqing Hua, Deborah Estrin, and Vitaly Shmatikov. 2020. How to backdoor federated learning. In *International conference on artificial intelligence and statistics*. PMLR, 2938–2948.

[5] Xinyun Chen, Chang Liu, Bo Li, Kimberly Lu, and Dawn Song. 2017. Targeted backdoor attacks on deep learning systems using data poisoning. *arXiv preprint arXiv:1712.05526*.

[6] Clement Fung, Chris J. M. Yoon, and Ivan Beschastnikh. 2020. Mitigating sybils in federated learning poisoning. (2020). https://arxiv.org/abs/1808.04866 arXiv: 1808.04866 [cs.LG].

[7] Micah Goldblum, Dimitris Tsipras, Chulin Xie, Xinyun Chen, Avi Schwarzschild, Dawn Song, Aleksander Mądry, Bo Li, and Tom Goldstein. 2023. Dataset security for machine learning: data poisoning, backdoor attacks, and defenses. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45, 2, 1563–1580. doi:10.1109/TPAMI.2022.3162397.

[8] Xueluan Gong, Yanjiao Chen, Qian Wang, and Weihan Kong. 2022. Backdoor attacks and defenses in federated learning: state-of-the-art, taxonomy, and future directions. *IEEE Wireless Communications*, 30, 2, 114–121.

[9] Briland Hitaj, Giuseppe Ateniese, and Fernando Perez-Cruz. 2017. Deep models under the gan: information leakage from collaborative deep learning. *arXiv preprint arXiv:1702.07464*.

[10] Peter Kairouz et al. 2021. Advances and open problems in federated learning. *Foundations and trends® in machine learning*, 14, 1–2, 1–210.

[11] Cody Lewis, Vijay Varadharajan, and Nasimul Noman. 2023. Attacks against federated learning defense systems and their mitigation. *Journal of Machine Learning Research*, 24, 30, 1–50. http://jmlr.org/papers/v24/22-0014.html.

[12] Zhuotao Lian, Chen Zhang, Kaixi Nan, and Chunhua Su. 2023. Spoil: sybil-based untargeted data poisoning attacks in federated learning. In *Network and System Security*. Shujun Li, Mark Manulis, and Atsuko Miyaji, (Eds.) Springer Nature Switzerland, Cham, 235–248. ISBN: 978-3-031-39828-5.

[13] liang5211314. 2024. Federated_backdoor_attack. https://github.com/liang5211314/federated_backdoor_attack. Accessed: 2025-07-05. (2024). https://github.com/liang5211314/federated_backdoor_attack.

[14] Pengrui Liu, Xiangrui Xu, and Wei Wang. 2022. Threats, attacks and defenses to federated learning: issues, taxonomy and perspectives. *Cybersecurity*, 5, 1, 4.

[15] Yingqi Liu, Shiqing Ma, Yousra Aafer, Wen-Chuan Lee, Juan Zhai, Weihang Wang, and Xiangyu Zhang. 2018. Trojaning attack on neural networks. In *25th Annual Network And Distributed System Security Symposium (NDSS 2018)*. Internet Soc.

[16] Zhijian Liu. 2019. Deep leakage from gradients. https://github.com/mit-han-lab/dlg. GitHub repository. (2019).

[17] Shrey Paul. 2025. Sybil-based untargeted data poisoning attack. https://github.com/ShreyPaul/untargeted-poisoning. GitHub repository. (2025).

[18] Amrita Roy Chowdhury, Chuan Guo, Somesh Jha, and Laurens van der Maaten. 2022. Eiffel: ensuring integrity for federated learning. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security* (CCS '22). Association for Computing Machinery, Los Angeles, CA, USA, 2535–2549. ISBN: 9781450394505. doi:10.1145/3548606.3560611.

[19] Bolun Wang, Yuanshun Yao, Shawn Shan, Huiying Li, Bimal Viswanath, Haitao Zheng, and Ben Y Zhao. 2019. Neural cleanse: identifying and mitigating backdoor attacks in neural networks. In *2019 IEEE symposium on security and privacy (SP)*. IEEE, 707–723.

[20] YunshiuanOAO. 2025. Sybil-based-data-poisoning-attacks-in-federated-learning-poc. https://github.com/YunshiuanOAO/Sybil-Based-Data-Poisoning-Attacks-in-Federated-Learning-Poc. GitHub repository. (2025).

[21] Bo Zhao, Konda Reddy Mopuri, and Hakan Bilen. 2020. Idlg: improved deep leakage from gradients. *arXiv preprint arXiv:2001.02610*.

[22] Jingjing Zheng, Kai Li, Xin Yuan, Wei Ni, and Eduardo Tovar. 2024. Detecting poisoning attacks on federated learning using gradient-weighted class activation mapping. In *Companion Proceedings of the ACM Web Conference 2024* (WWW '24). Association for Computing Machinery, Singapore, Singapore, 714–717. ISBN: 9798400701726. doi:10.1145/3589335.3651490.

[23] Changxun Zhu, Qilong Wu, Lingjuan Lyu, and Shibei Xue. 2025. Sybil-based virtual data poisoning attacks in federated learning. (2025). https://arxiv.org/abs/2505.09983 arXiv: 2505.09983 [cs.CR].

[24] Ligeng Zhu, Zhijian Liu, and Song Han. 2019. Deep leakage from gradients. In *Advances in Neural Information Processing Systems*.

A Reconstructed Image

Figure 7 shows the entire process of retrieving the image during the DLG attack over 290 epochs. The sequence demonstrates how the DLG attack refines a random initialization into an accurate reconstruction of the original input.

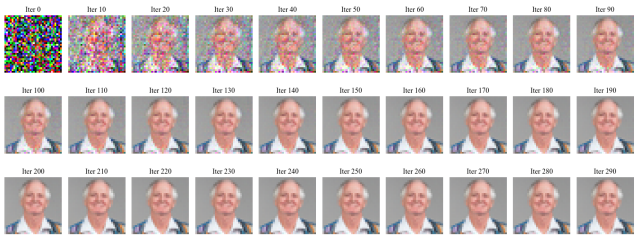

Figure 7: Evolution of the reconstructed image over optimization iterations.

B DLG Mathematical Formulation

The Deep Leakage from Gradients attack exploits the fundamental mathematical relationship between gradients and their originating data. Consider a machine learning model F parameterized by weights W. For a private input-label pair $(\mathbf{x}, \mathbf{y})$, the gradients ∇W computed during backpropagation are defined as[24]:

$$\nabla W = \frac{\partial \ell(F(\mathbf{x}, W), \mathbf{y})}{\partial W} \tag{2}$$

The critical insight underlying DLG is that these gradients ∇W constitute a *deterministic function* of the original data $(\mathbf{x}, \mathbf{y})$. Consequently, reconstructing private data from gradients transforms into solving an **inverse optimization problem**:

(1) **Dummy initialization**: Generate initial approximations $(\mathbf{x}', \mathbf{y}')$ through random sampling:

$$\mathbf{x}' \sim \mathcal{N}(0, \sigma^2), \quad \mathbf{y}' \sim \text{Uniform}(0, 1)^C$$

where $\mathcal{N}$ denotes Gaussian noise and C is the number of classes.

(2) **Dummy gradient computation**: Using the current model weights W, compute pseudo-gradients from dummy data:

$$\nabla W' = \frac{\partial \ell(F(\mathbf{x}', W), \mathbf{y}')}{\partial W} \tag{3}$$

(3) **Gradient matching optimization**: Minimize the L_2 distance between real and dummy gradients:

$$\min_{\mathbf{x}', \mathbf{y}'} \left\| \nabla W' - \nabla W \right\|_2^2 \tag{4}$$

C Mathematical Formulation of Backdoor Attack

The standard classification loss is:

$$\mathcal{L}_{\text{task}} = \mathbb{E}_{(x,y)\sim\mathcal{D}} \left[\ell(f_\theta(x), y) \right]$$

where θ denotes model parameters and ℓ is typically the cross-entropy loss.

Poisoned samples are constructed as:

$$x_{\text{poison}} = x \oplus \delta, \quad y_{\text{target}} \neq y$$

where δ represents a trigger pattern and $\oplus$ denotes element-wise addition.

A malicious client optimizes:

$$\min_\theta \underbrace{\mathcal{L}_{\text{task}}}_{\text{main task}} + \lambda \underbrace{\mathbb{E}_{x\sim\mathcal{D}_{\text{poison}}} \left[\ell(f_\theta(x \oplus \delta), y_{\text{target}}) \right]}_{\text{backdoor task}}$$

where λ is a balancing factor controlling the trade-off between accuracy and backdoor effectiveness.